



Instituto Técnico Ricaldone

Desarrollo de Software

“PTC”

Docentes:

Emerson González

Alumnos:

Edgar Alfredo Gómez Gonzales 20240109

Gerardo Adriel Avalos Flores 20230417

Dimas Samuel Argueta Bonilla 20240434

María Renee Celada Meléndez 20240162

Sara Astrid Argueta López 20240539

Índice

Introducción	3
2.Tecnologías y herramientas	4
2.1Tecnologías	4
2.2 Herramientas	5
3. Estructura de la base de datos	7
3.1 Modelo de relacional	7
3.2 Script de la base de datos	8
4.Diccionario de datos	15
5.Arquitectura del software.....	16
6. Estructura del proyecto	17
7.Diseño de la aplicación	18
Paleta de colores.....	18
Aplicación en los Diferentes Elementos	19
8.Buenas prácticas de desarrollo.....	19
Estándares de programación en C#	19
9. Requerimientos de hardware y software.....	31
Localmente	31
Servidor	32
10. Instalación y configuración.....	33

Manual Técnico

INTRODUCCIÓN

En el presente trabajo técnico describe el desarrollo de una aplicación de escritorio diseñada para la optimización, la gestión administrativa y operativa de un salón de Belleza. En este trabajo se espera realizar una mejora los procesos internos relacionados al control de inventario y la organización de citas, así como el control de clientes.

En la actualidad el negocio tiene un sistema digital eficiente, lo que ocasiona un orden en el inventario y genera cero complicaciones, en la programación de clientes, con esto la finalidad fue solucionar deficiencias mediante un entorno intuitivo y de fácil uso, que permita a los usuarios ingresar y administrar la información de manera estructurada.

Este documento servirá como guía para comprender los objetivos, y funcionalidades y alcances del software, así como los criterios técnicos empleados en su diseño y desarrolla. Además, proporciona los lineamientos necesarios para asegurar que la herramienta sea implementada correctamente y cumpla con los requerimientos planteados.

2.Tecnologías y herramientas

2.1Tecnologías

- C# (.NET **Framework 4.8**)

Uso: Para la construcción de la aplicación de escritorio.

Sitio web:

<https://learn.microsoft.com/es-es/dotnet/csharp/>

- Windows Forms

Uso: Para construir interfaces de usuario intuitivas y adaptadas al personal del salón.

Sitio web:

<https://learn.microsoft.com/es-es/dotnet/desktop/winforms/>

- SQL Server Expres

Uso: Asegura el almacenamiento estructurado de clientes, citas, inventario, etc.

Sitio web:

<https://www.microsoft.com/es-es/sql-server/sql-server-downloads>

- Azure SQL Database

Uso: será especialmente útil si se desea implementar una solución en la nube.

Sitio web:

<https://azure.microsoft.com/es-es/products/azure-sql/database>

- HTML5 y CSS3

Uso: Para elaborar la página web de apoyo.

Sitio web:

<https://developer.mozilla.org/es/docs/Web/HTML>

2.2 Herramientas

- **Visual Studio Community 2022**

Uso: Para programar la aplicación de escritorio en C#

Sitio web:

<https://visualstudio.microsoft.com/es/>

- **Visual Studio Code**

Uso: editor ligero para la edición de código web

Sitio web:

<https://code.visualstudio.com/>

- **SQL Server Management Studio (SSMS)**

Uso: Para la creación la administración, monitoreo y creación de consultas en la base de datos.

Sitio web:

<https://learn.microsoft.com/es-es/sql/ssms/download-sql-server-management-studio-ssms>

- **Figma**

Uso: Para la creación del diseño y estructura del sistema.

Sitio web:

<https://www.figma.com/>

- Draw.io

Uso: La utilizamos para la elaboración de diagramas

Sitio web:

<https://app.diagrams.net/>

- Lucidchart

Uso: La utilizamos para la elaboración de diagramas

Sitio web:

<https://www.lucidchart.com/>

3. Estructura de la base de datos

3.1 Modelo de relacional

- Diagrama de dominio de la base de datos el proyecto.

https://drive.google.com/file/d/1DBpOAEkLtVtWTaTwi_ZZwZxo33HW2oZu/view?ts=68814e1b&huid=m8Fz25RkudD5VxAc2w5yFQ

3.2 Script de la base de datos

```
1 CREATE DATABASE Ligia_SalonPTC2;
2 GO
3 USE Ligia_SalonPTC2;
4 go
5
6
7 --Tabla de roles--
8 CREATE TABLE roles (
9     id_Rol int primary key identity,
10    nombre nvarchar(50) unique not null
11 );
12 GO
13
14
15 -- 1. Tabla de usuarios
16 CREATE TABLE usuario (
17     id_Usuario INT IDENTITY(1,1) PRIMARY KEY,
18     nombre_Usuario VARCHAR(50) UNIQUE NOT NULL,
19     clave VARCHAR(100) NOT NULL,
20     id_Rol INT FOREIGN KEY (id_Rol)
21     REFERENCES roles (id_Rol) on delete cascade
22 );
23 GO
24
25 -- 2. Tabla de clientes
26 CREATE TABLE cliente (
27     id_Cliente INT IDENTITY(1,1) PRIMARY KEY,
28     nombre_Cliente NVARCHAR(100) NOT NULL,
29     correo NVARCHAR(100) UNIQUE NOT NULL,
30     telefono NVARCHAR(50) NOT NULL,
31     direccion NVARCHAR(150) NOT NULL,
32     id_Usuario_int
33     FOREIGN KEY (id_Usuario_) REFERENCES usuario(id_Usuario) ON DELETE CASCADE
34 );
35 GO
```



```

-- 3. Tabla de inventario
CREATE TABLE inventario (
    id_Inventario INT IDENTITY(1,1) PRIMARY KEY,
    nombre NVARCHAR(50) UNIQUE NOT NULL,
    descripcion NVARCHAR(100),
    precio DECIMAL(10,2) NOT NULL,

);
GO

-- 4. Tabla de servicio
CREATE TABLE servicio (
    id_Servicio INT IDENTITY(1,1) PRIMARY KEY,
    nombre_Servicio NVARCHAR(50) NOT NULL,
    descripcion_Servicio NVARCHAR(100),
    precio DECIMAL(10,2) NOT NULL

);
GO

-- 5. Tabla de cita
CREATE TABLE cita (
    id_Cita INT IDENTITY(1,1) PRIMARY KEY,
    id_Cliente INT NOT NULL,
    id_Servicio INT NOT NULL,
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    FOREIGN KEY (id_Cliente) REFERENCES cliente(id_Cliente) ON DELETE CASCADE,
    FOREIGN KEY (id_Servicio) REFERENCES servicio(id_Servicio) ON DELETE CASCADE

);
GO

```

```

67
68 -- 6. Tabla de factura
69 CREATE TABLE factura (
70     id_Factura INT IDENTITY(1,1) PRIMARY KEY,
71     numero_Factura VARCHAR(30) UNIQUE NOT NULL,
72     fecha DATE NOT NULL,
73     id_Cliente INT NOT NULL,
74     total_SinIVA DECIMAL(10,2) NOT NULL,
75     total_ConIVA DECIMAL(10,2) NOT NULL,
76     iva_Total DECIMAL(10,2) NOT NULL,
77     FOREIGN KEY (id_Cliente) REFERENCES cliente(id_Cliente) ON DELETE CASCADE
78 );
79 GO
80
81 -- 7. Tabla detalle_producto
82 CREATE TABLE detalle_producto (
83     id_Detalle_Producto INT IDENTITY(1,1) PRIMARY KEY,
84     id_Factura INT NOT NULL,
85     id_Inventario INT NOT NULL,
86     cantidad INT NOT NULL,
87     precio_Unitario DECIMAL(10,2) NOT NULL,
88     incluye_IVA BIT NOT NULL,
89     subtotal DECIMAL(10,2) NOT NULL,
90     FOREIGN KEY (id_Factura) REFERENCES factura(id_Factura) ON DELETE CASCADE,
91     FOREIGN KEY (id_Inventario) REFERENCES inventario(id_Inventario) ON DELETE CASCADE
92 );
93 GO
94
95 -- 8. Tabla detalle_servicio
96 CREATE TABLE detalle_servicio (
97     id_Detalle_Servicio INT IDENTITY(1,1) PRIMARY KEY,
98     id_Factura INT NOT NULL,
99     id_Servicio INT NOT NULL,
100     cantidad INT NOT NULL,
101     precio_Unitario DECIMAL(10,2) NOT NULL,
102     subtotal DECIMAL(10,2) NOT NULL,
103     FOREIGN KEY (id_Factura) REFERENCES factura(id_Factura) ON DELETE CASCADE,
104     FOREIGN KEY (id_Servicio) REFERENCES servicio(id_Servicio) ON DELETE CASCADE
105
106

```

```

111
112 -- Datos para inventario
113 INSERT INTO inventario VALUES
114 ('Shampoo Hidratante', 'Para cabello seco', 4.50),
115 ('Acondicionador Keratina', 'Repara y suaviza', 5.25),
116 ('Tinte Rojo', 'Color intenso 30 días', 6.00),
117 ('Crema Capilar', 'Antifrizz', 3.50),
118 ('Serum Nutritivo', 'Brillo inmediato', 8.00),
119 ('Gel Fijador', 'Control extremo', 2.75),
120 ('Spray Capilar', 'Volumen natural', 4.25),
121 ('Tratamiento Reconstrucción', 'Uso profesional', 9.00),
122 ('Mascarilla Capilar', 'Con aceite de coco', 6.75),
123 ('Tinte Azul', 'Color frío y duradero', 6.00),
124 ('Shampoo Anticaspa', 'Con menta refrescante', 5.00),
125 ('Tinte Rubio', 'Efecto natural', 6.50),
126 ('Aceite Capilar', 'Reparación profunda', 7.00),
127 ('Laca Fijadora', 'Fijación invisible', 3.50),
128 ('Ampollas de Tratamiento', 'Uso único', 2.00);
129 GO
130
131 -- Datos para clientes
132 --Tema del id_usuario
133 --por el tema de la contra encriptada no hay usuario creado todavía
134 INSERT INTO cliente VALUES
135 ('Ana López', 'ana.lopez@mail.com', '7777-0001', 'Col. Escalón', 2),
136 ('Luis Mejía', 'luis.mejia@mail.com', '7777-0002', 'San Benito', 2),
137 ('María Torres', 'maria.torres@mail.com', '7777-0003', 'Antiguo Cuscatlán', 2),
138 ('Carlos Gómez', 'carlos.gomez@mail.com', '7777-0004', 'Santa Tecla', 2),
139 ('Andrea Hernández', 'andrea.hdz@mail.com', '7777-0005', 'Zaragoza', 2),
140 ('Verónica Castro', 'vero.castro@mail.com', '7777-0006', 'Col. Médica', 2),
141 ('Santiago Varela', 'santi.varela@mail.com', '7777-0007', 'Merliot', 2),
142 ('Carmen Reyes', 'carmen.reyes@mail.com', '7777-0008', 'Soyapango', 2),
143 ('Ricardo Molina', 'ricardo.m@mail.com', '7777-0009', 'Ilopango', 2),
144 ('Patricia Díaz', 'paty.diaz@mail.com', '7777-0010', 'Col. San Francisco', 2),
145 ('Diego Campos', 'diego.campos@mail.com', '7777-0011', 'Cuscatancingo', 2),
146 ('Valeria Peralta', 'valeria.p@mail.com', '7777-0012', 'Col. San Mateo', 2),

```

```

151
152 -- Datos para servicios
153 INSERT INTO servicio VALUES
154 ('Corte de cabello', 'Corte estándar unisex', 7.00),
155 ('Tinte completo', 'Aplicación de tinte personalizado', 15.00),
156 ('Planchado', 'Planchado profesional', 10.00),
157 ('Tratamiento capilar', 'Nutrición profunda', 12.50),
158 ('Peinado especial', 'Para eventos o fotos', 8.50),
159 ('Mechas', 'Efecto degradado o balayage', 20.00),
160 ('Mascarilla capilar', 'Hidratación intensa', 6.00),
161 ('Limpieza capilar', 'Con exfoliante y menta', 5.00),
162 ('Secado', 'Secado con cepillo', 4.50),
163 ('Desrizado', 'Eliminación de ondas', 15.50),
164 ('Corte infantil', 'Niños hasta 10 años', 5.50),
165 ('Tinte por zonas', 'Raíces o puntas', 9.00),
166 ('Ondulado temporal', 'Con pinzas térmicas', 11.00),
167 ('Limpieza de cuero cabelludo', 'Anticaspa intensivo', 6.50),
168 ('Servicio express', 'Peinado rápido', 3.50);
169 GO
170
171
172
173
174 -- Datos para citas
175 --Lo mismo para citas
176 INSERT INTO cita VALUES
177 (1, 1, '2025-07-24', '10:00'),
178 (2, 2, '2025-07-24', '10:45'),
179 (3, 3, '2025-07-24', '11:30'),
180 (4, 4, '2025-07-24', '12:00'),
181 (5, 5, '2025-07-24', '12:30'),
182 (6, 6, '2025-07-25', '09:00'),
183 (7, 7, '2025-07-25', '09:45'),
184 (8, 8, '2025-07-25', '10:15'),
185 (9, 9, '2025-07-25', '11:00'),
186 (10, 10, '2025-07-25', '11:45'),
187 (11, 11, '2025-07-25', '13:00'),
188 (12, 12, '2025-07-26', '08:30'),

```

%  No se encontraron problemas.

```

212 GO
213
214 -- Detalle de servicios
215 --lo mismo
216 INSERT INTO detalle_servicio VALUES
217 (1, 1, 1, 7.00, 7.00), -- Corte de cabello
218 (2, 2, 1, 15.00, 15.00), -- Tinte completo
219 (3, 3, 1, 10.00, 10.00), -- Planchado
220 (4, 6, 1, 20.00, 20.00), -- Mechass
221 (5, 5, 1, 8.50, 8.50), -- Peinado especial
222 (6, 6, 1, 20.00, 20.00), -- Mechass
223 (7, 7, 1, 6.00, 6.00), -- Mascarilla capilar
224 (7, 9, 1, 4.50, 4.50), -- Secado
225 (8, 10, 1, 15.50, 15.50), -- Desrizado
226 (9, 11, 1, 5.50, 5.50), -- Corte infantil
227 (10, 8, 1, 5.00, 5.00), -- Limpieza capilar
228 (11, 12, 1, 9.00, 9.00), -- Tinte por zonas
229 (12, 13, 1, 11.00, 11.00), -- Ondulado temporal
230 (12, 9, 1, 4.50, 4.50), -- Secado
231 (13, 14, 1, 6.50, 6.50), -- Limpieza cuero cabelludo
232 (13, 1, 1, 7.00, 7.00), -- Corte de cabello
233 (14, 5, 1, 8.50, 8.50), -- Peinado especial
234 (15, 3, 1, 10.00, 10.00); -- Planchado
235 GO
236
237 -- Detalle de productos vendidos
238 --lo mismo
239 INSERT INTO detalle_producto VALUES
240 (1, 1, 1, 4.50, 1, 4.50), -- Shampoo Hidratante
241 (2, 2, 1, 5.25, 1, 5.25), -- Acondicionador Keratina
242 (3, 4, 1, 3.50, 1, 3.50), -- Crema Capilar
243 (4, 1, 1, 4.50, 1, 4.50), -- Shampoo Hidratante
244 (5, 3, 1, 6.00, 1, 6.00), -- Tinte Rojo
245 (6, 5, 1, 8.00, 1, 8.00), -- Serum Nutritivo
246 (7, 6, 1, 2.75, 1, 2.75), -- Gel Fijador
247 (8, 7, 1, 4.25, 1, 4.25), -- Spray Capilar
248 (9, 9, 1, 6.75, 1, 6.75), -- Mascarilla Capilar
249 (10, 11, 1, 3.50, 1, 3.50), -- Gel Fijador

```

 No se encontraron problemas

```

260 SELECT * FROM usuario;
261 Go
262 SELECT * FROM cliente;
263 Go
264 SELECT * FROM inventario;
265 Go
266 SELECT * FROM servicio;
267 Go
268 SELECT * FROM cita;
269 Go
270 SELECT * FROM factura;
271 Go
272 SELECT * FROM detalle_producto;
273 Go
274 SELECT * FROM detalle_servicio;
275 Go
276 -- Ver lo que compró cada cliente:
277 SELECT f.numero_Factura, c.nombre_Cliente, f.fecha, s.nombre_Servicio, ds.cantidad, ds.subtotal
278 FROM detalle_servicio ds
279 JOIN factura f ON ds.id_Factura = f.id_Factura
280 JOIN servicio s ON ds.id_Servicio = s.id_Servicio
281 JOIN cliente c ON f.id_Cliente = c.id_Cliente;
282 Go
283 -- Ver los productos por factura:
284 SELECT f.numero_Factura, c.nombre_Cliente, f.fecha, i.nombre, dp.cantidad, dp.subtotal
285 FROM detalle_producto dp
286 JOIN factura f ON dp.id_Factura = f.id_Factura
287 JOIN inventario i ON dp.id_Inventario = i.id_Inventario
288 JOIN cliente c ON f.id_Cliente = c.id_Cliente;
289 Go

```

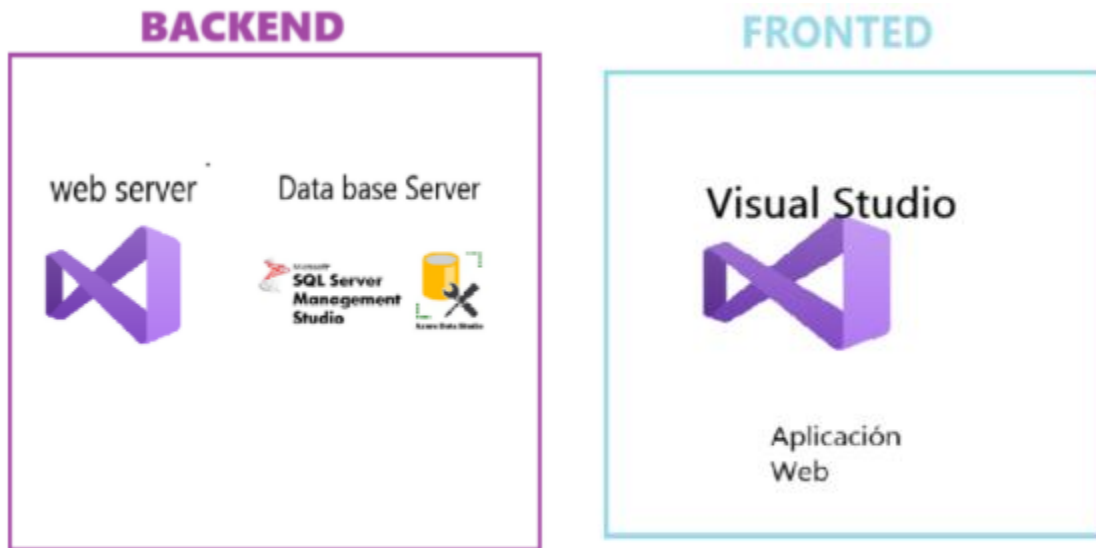
<https://drive.google.com/file/d/1rzPuTadhs6NWUPxHjLErk28F5Qgw425/>

[view?usp=sharing](#)

4.Diccionario de datos

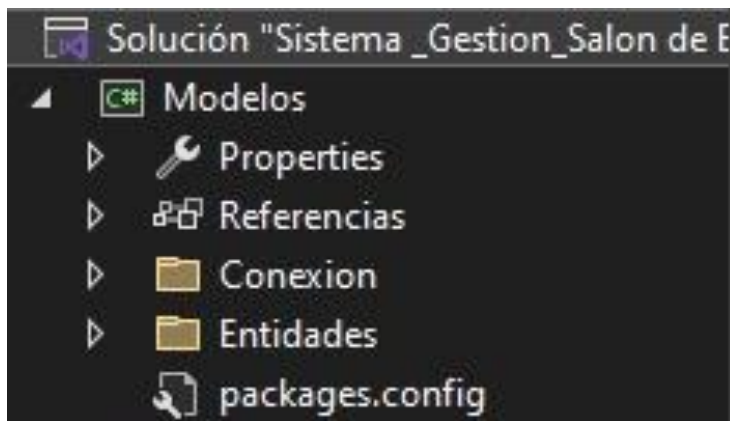
Tabla	Campo	Tipo de Dato	Longitud	Restricciones	Descripción
Usuario	id_usuario	INT	11	PRIMARY KEY	Identificador único del usuario
Usuario	nombre	VARCHAR	100	NOT NULL	Nombre completo del usuario
Usuario	correo	VARCHAR	150	UNIQUE, NOT NULL	Correo electrónico del usuario
Usuario	contraseña	VARCHAR	255	NOT NULL	Contraseña encriptada del usuario
Usuario	fecha	DATE	-	NOT NULL	Fecha de registro en el sistema

5.Arquitectura del software

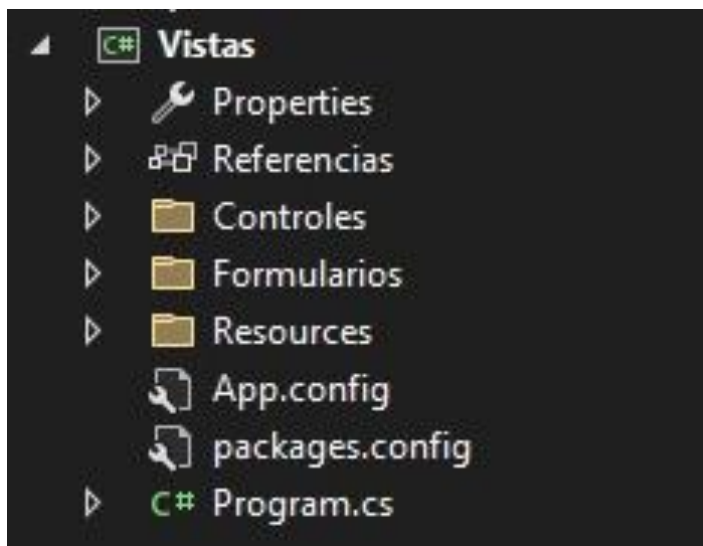


6. Estructura del proyecto

Estructura detallada de la carpeta Models



Estructura detallada de la carpeta Views

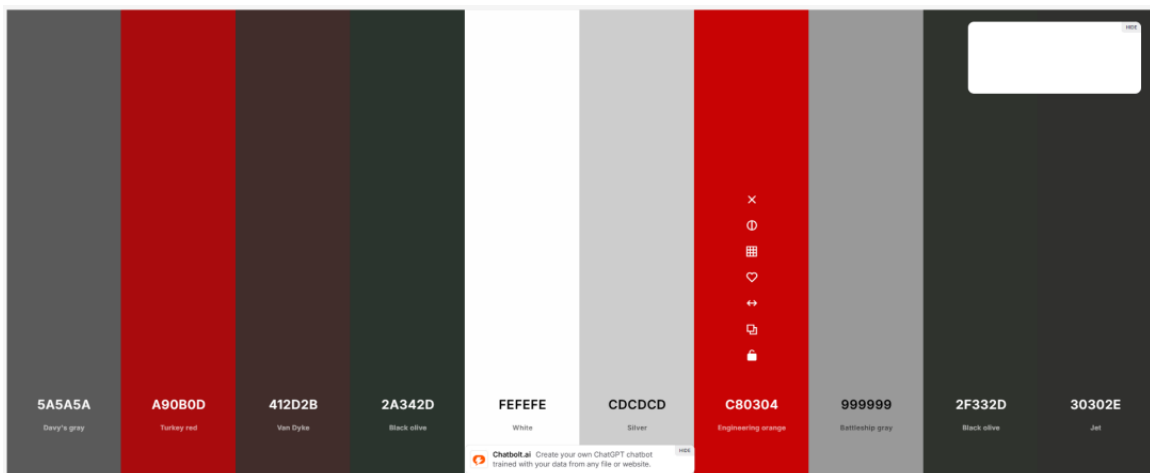


7.Diseño de la aplicación

Paleta de Colores

En el proyecto *Ligia salón* se ha empleado una paleta de colores basada en tonalidades rojas, grises y tierra, con el objetivo de mantener una estética coherente y adecuada a la marca desarrollada. Esta selección está en sintonía con el logotipo y alineada con la identidad visual corporativa del proyecto. La paleta fue elegida considerando tanto el diseño del logo como la intención de ofrecer una experiencia visual agradable y moderna.

Paleta de colores



Aplicación en los Diferentes Elementos

En *Ligia salón*, la paleta de colores basada en tonos rojos y grises se aplica de manera estratégica en los distintos elementos de la aplicación, con el objetivo de garantizar una experiencia de usuario coherente, estética y agradable.

A continuación, se presentan algunas imágenes que ilustran cómo se emplea esta paleta en el entorno visual y en los diversos componentes de la interfaz.

8. Buenas prácticas de desarrollo

Estándares de programación en C#

Los estándares de programación en C# son un conjunto de pautas y convenciones que seguimos para escribir código consistente, legible y fácil de mantener en el proyecto *Ligia's Salon*.

A continuación, se describen los principales estándares aplicados que reflejan claramente el propósito de cada elemento. Esto facilita la comprensión del código por parte de otros desarrolladores y mejora la mantenibilidad del sistema.:

1. Nomenclatura de Variables, Controles y Métodos

Prefijos para controles de WinForms:

- txt para TextBox = txtNombreCliente
- btn para Button = btnGuardarFactura
- cmb para ComboBox = cmbServicios
- dgv para DataGridView = dgvInventario

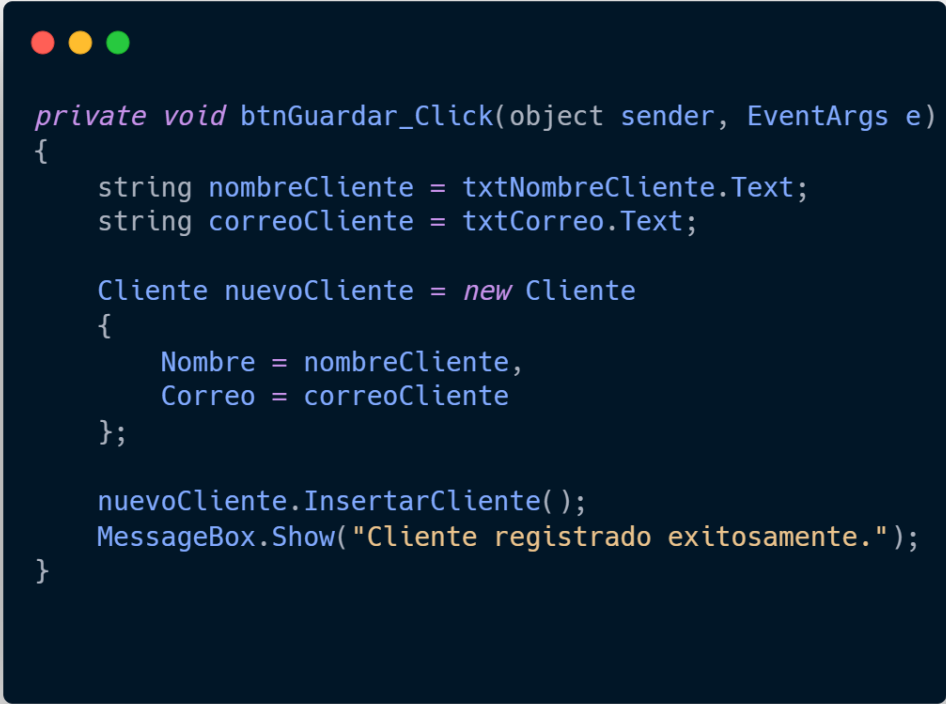
•PascalCase para clases, métodos públicos y propiedades:

- Clase: Cliente
- Método: GuardarFactura()
- Propiedad: TotalConIva

• camelCase para variables locales y parámetros:

- numeroFactura
- subtotalSinIva

Ejemplo:



```
private void btnGuardar_Click(object sender, EventArgs e)
{
    string nombreCliente = txtNombreCliente.Text;
    string correoCliente = txtCorreo.Text;

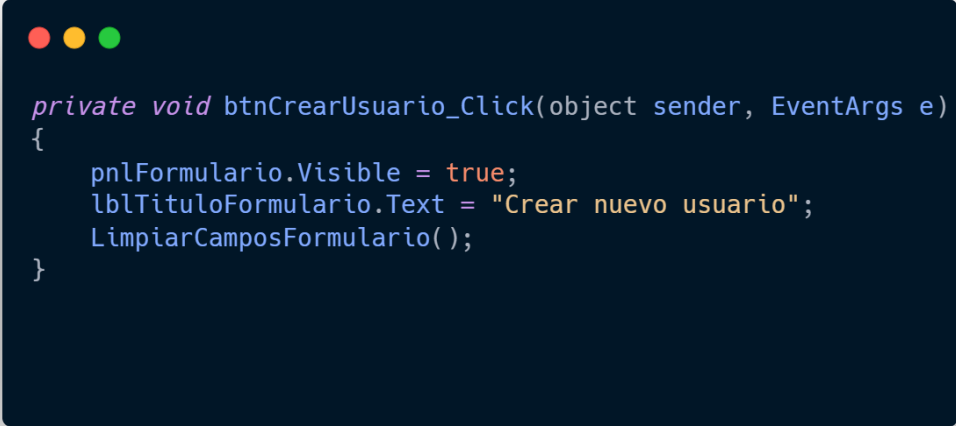
    Cliente nuevoCliente = new Cliente
    {
        Nombre = nombreCliente,
        Correo = correoCliente
    };

    nuevoCliente.InsertarCliente();
    MessageBox.Show("Cliente registrado exitosamente.");
}
```

Nombres Significativos

Los nombres de variables, controles y métodos en el proyecto **Ligia's Salon** son claros e indican su propósito específico. Esta práctica mejora la legibilidad del código y facilita su mantenimiento.

Ejemplo:



```
private void btnCrearUsuario_Click(object sender, EventArgs e)
{
    pnlFormulario.Visible = true;
    lblTituloFormulario.Text = "Crear nuevo usuario";
    LimpiarCamposFormulario();
}
```

2. Indentación

La indentación se refiere al uso de espacios al inicio de una línea de código para organizar visualmente el flujo y la jerarquía del código.

- **Consistente:** Se emplea una indentación de 4 espacios por nivel, garantizando la legibilidad del código.

Ejemplo:



```
private void btnGuardar_Click(object sender, EventArgs e)
{
    string nombreCliente = txtNombreCliente.Text;
    string correoCliente = txtCorreo.Text;

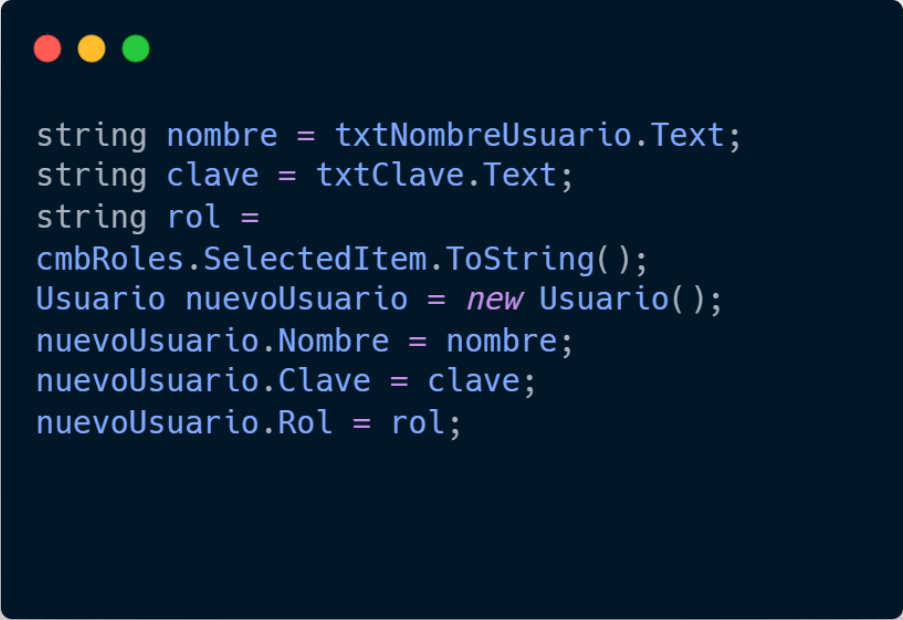
    Cliente nuevoCliente = new Cliente
    {
        Nombre = nombreCliente,
        Correo = correoCliente
    };

    nuevoCliente.InsertarCliente();
    MessageBox.Show("Cliente registrado exitosamente.");
}
```

3. Punto y Coma

- La inclusión de punto y coma (;) al final de cada instrucción es esencial en C#. Esta práctica evita errores de compilación y garantiza que el código se ejecute correctamente. En el proyecto, todas las declaraciones y sentencias finalizan con punto y coma, siguiendo la sintaxis del lenguaje.

Ejemplo:

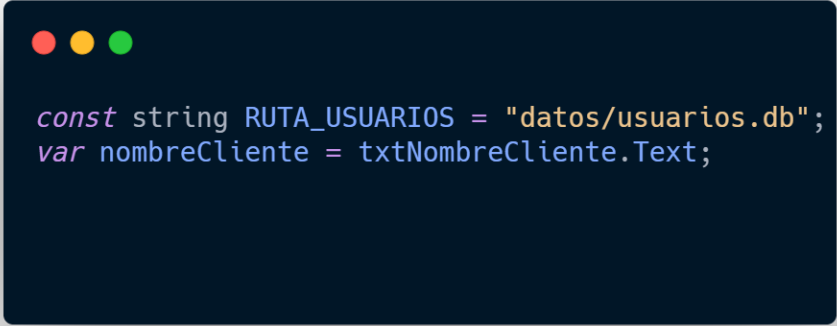


```
string nombre = txtNombreUsuario.Text;
string clave = txtClave.Text;
string rol =
cmbRoles.SelectedItem.ToString();
Usuario nuevoUsuario = new Usuario();
nuevoUsuario.Nombre = nombre;
nuevoUsuario.Clave = clave;
nuevoUsuario.Rol = rol;
```

4. Declaración de Variables

- Uso de const y var: Se utilizó const para variables cuyo valor permaneció inmutable, y var para aquellas que cambiaron durante la ejecución. No se utilizó ningún

tipo de declaración sin tipo definido. Ejemplo:



```
const string RUTA_USUARIOS = "datos/usuarios.db";  
var nombreCliente = txtNombreCliente.Text;
```

5. Evitar el Uso de Variables Globales

- Las variables globales deben minimizarse para evitar conflictos de nombres y mejorar el mantenimiento del código. En C#, se recomienda encapsular los datos dentro de clases y utilizar propiedades o métodos para acceder a ellos.

6. Evitar la Escritura en Línea

- **Legibilidad:** Se evita declarar múltiples instrucciones en una sola línea para mejorar la lectura del código.

Ejemplo:



```
if (chkRecepcionista.Checked)
{
    rolAsignado = "Recepcionista";
}
else
{
    rolAsignado = "Estilista";
}
```

7. Manejo de Errores

- **Notificación de errores:** Se utilizó `MessageBox.Show(...)` para informar al usuario sobre errores ocurridos durante la ejecución. Esta técnica permitió mostrar mensajes claros en tiempo de ejecución.

Ejemplo:



```
if (!resultado)
{
    MessageBox.Show("Ocurrió un error al intentar actualizar el registro.", "Error",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

Conclusión:

La adopción de estos estándares garantizó un código claro, legible y mantenible, lo que facilitó la colaboración entre desarrolladores y mejoró la eficiencia en la detección y corrección de errores. De esta manera, se promueve un entorno de desarrollo más ordenado y sostenible dentro del sistema además de permitir una comunicación directa con el usuario ante eventos inesperados, lo que facilitó la identificación de errores durante el uso de la aplicación y contribuyó a mantener la claridad en el flujo de ejecución

Estándares de programación en MySQL

El objetivo de este documento es establecer una serie de lineamientos y buenas prácticas para la programación en MySQL, con el fin de garantizar la calidad, claridad y facilidad de mantenimiento del código en el proyecto Ligia's Salon PTC.

1. Nombres de Tablas y Campos:

- **Tablas:** Deben nombrarse en minúsculas y en plural.

Ejemplo:

```

CREATE TABLE usuario (
  id_Usuario INT IDENTITY(1,1) PRIMARY KEY,
  nombre_Usuario VARCHAR(50) UNIQUE NOT NULL,
  clave VARCHAR(100) NOT NULL,
  id_Rol INT FOREIGN KEY (id_Rol)
  REFERENCES roles (id_Rol) on delete cascade
);
GO

```

- **Campos:** Deben nombrarse en minúsculas, en singular y separados por guiones bajos.

Ejemplo:

```

id_Cliente INT IDENTITY(1,1) PRIMARY KEY,

```

2. Nomenclatura de Variables

Dado que en este proyecto no se ha adoptado un esquema de prefijos o sufijos para las variables, todas ellas deben describirse en el glosario de variables. En dicho glosario se debe consignar el nombre exacto tal como aparece en el código, el tipo de dato SQL, su ámbito (parámetro o variable interna) y una breve descripción de su propósito. Los parámetros se documentan en la cabecera de cada procedimiento, función o trigger; las variables internas, mediante la sentencia DECLARE al inicio del cuerpo. Este listado debe mantenerse actualizado cada vez que se agregue, elimine o modifique cualquier variable o parámetro, garantizando así la trazabilidad, legibilidad y facilidad de mantenimiento del código.

3. Nombres de procedimientos almacenados, vistas y funciones

En el sistema, no se han implementado procedimientos almacenados, vistas ni funciones dentro del script actual. Por lo tanto:

- **Procedimientos almacenados:** *No se utilizan en este proyecto.* No hay ninguna lógica encapsulada en procedimientos SQL. Todas las operaciones se realizan directamente mediante consultas estándar y relaciones entre tablas.
- **Vistas:** *No se han definido vistas.* La visualización de datos se realiza mediante consultas directas, como las que permiten ver productos por factura o servicios adquiridos por cliente.
- **Funciones:** *No se han creado funciones definidas por el usuario.* El sistema no requiere cálculos personalizados encapsulados en funciones SQL, ya que los cálculos como subtotales, IVA y totales se almacenan directamente en las tablas correspondientes.

Nota técnica: Aunque el manual de referencia sugiere el uso de procedimientos con nombres en plural y parámetros con prefijo p_, en este proyecto no aplica, ya que no se ha utilizado esa estructura. En su lugar, se prioriza un diseño relacional claro y completo, con integridad referencial y datos precalculados en las tablas de detalle y factura.

4. Comentarios

Los comentarios en el código SQL cumplen una función esencial: documentar el propósito de cada bloque, facilitar su comprensión y mantener una estructura clara y ordenada. Esta práctica permite que cualquier desarrollador pueda entender con precisión qué se está creando, por qué se hace y cómo se relaciona con el resto del sistema.

- **Tablas:** Se indica la función de cada tabla (usuarios, clientes, servicios, etc.).
- **Inserciones:** Se explican decisiones técnicas, como omisiones intencionales o lógica repetida.
- **Consultas:** Se aclara qué datos se extraen y con qué fin.
- **Estructura:** El script está organizado por bloques comentados, lo que facilita su lectura, validación y defensa técnica.

Ejemplo:

```
-- Consulta que muestra los servicios adquiridos por cada cliente junto con su factura
SELECT
  f.numero_Factura,           -- Número único de la factura
  c.nombre_Cliente,          -- Nombre del cliente que realizó la compra
  f.fecha,                   -- Fecha en que se emitió la factura
  s.nombre_Servicio,         -- Nombre del servicio adquirido
  ds.cantidad,               -- Cantidad de veces que se aplicó el servicio
  ds.subtotal                -- Subtotal correspondiente al servicio
FROM detalle_servicio ds
JOIN factura f ON ds.id_Factura = f.id_Factura
JOIN servicio s ON ds.id_Servicio = s.id_Servicio
JOIN cliente c ON f.id_Cliente = c.id_Cliente;
```

Conclusión

La implementación de estos estándares en el sistema garantiza que el código SQL se mantenga limpio, claro y técnicamente defendible. Al aplicar buenas prácticas como la documentación mediante comentarios, la organización por bloques funcionales y la validación estructural, se fortalece la calidad del desarrollo y se facilita la colaboración entre

desarrolladores. Esto no solo mejora la comprensión del sistema, sino que también asegura su sostenibilidad y escalabilidad a largo plazo.

9. Requerimientos de hardware y software

Localmente

Cuando se habla de requerimientos de hardware y software en entorno local, se hace referencia a las condiciones óptimas para una ejecución satisfactoria del sistema, desarrollado en C# con Windows Forms y SQL Server, y ejecutado directamente en la máquina del usuario sin conexión a servidores externos ni servicios en la nube.

Hardware mínimo:

- 4 GB de RAM (8 GB recomendados para mayor fluidez)
- Procesador Intel Core i3 (8ª generación) o superior
- 300 MB de espacio disponible para la aplicación, más espacio adicional

para la base de datos local

Software mínimo:

1. **Sistema operativo**

Windows 10 o Windows 11 (no compatible con macOS ni distribuciones Linux)

2. **Entorno de desarrollo integrado (IDE)**

Visual Studio 2022 (utilizado únicamente para desarrollo, mantenimiento o modificación

técnica del sistema)

No se requieren plugins adicionales como Prettier, ESLint ni control de Git

3. **Lenguajes y tecnologías utilizadas**

- C# como lenguaje principal
- Windows Forms como tecnología de interfaz gráfica
- SQL Server Express o LocalDB como motor de base de datos
- Conexión mediante ADO.NET para el acceso a datos
- .NET Framework 4.7.2 o superior

Servidor

Este sistema **no requiere servidor web ni infraestructura en la nube**. Toda la ejecución, almacenamiento y gestión de datos se realiza de forma local en el equipo del usuario.

- **CPU (Procesador)**

No se requiere arquitectura de servidor. Basta con un procesador de uso general como Intel Core i3 o superior

- **Memoria RAM**

4 GB como mínimo; 8 GB recomendados para un rendimiento óptimo

- **Almacenamiento (Disco Duro)**

Espacio suficiente para la aplicación y la base de datos local. No se requiere SSD ni configuración RAID

- **Ancho de banda de red**

No se requiere conexión a red ni acceso a internet para el funcionamiento del sistema

- **Backup y redundancia**

No se implementa sistema de backup automático ni almacenamiento en la nube. La base de datos permanece en el equipo del usuario, y se recomienda realizar copias manuales periódicas según criterio personal

10. Instalación y configuración

Este sistema de escritorio puede ser instalado en cualquier equipo con sistema operativo Windows que cumpla los requisitos mínimos. A continuación, se describen los pasos necesarios para realizar la instalación en una nueva máquina.

1. Instalación en entorno local

Requisitos previos

Antes de comenzar la instalación, asegúrese de tener los siguientes componentes instalados:

- **Entorno de desarrollo:** Visual Studio 2022
- **Gestor de base de datos:** SQL Server Express o SQL Server Management Studio (SSMS)

- **Framework:** .NET Framework 4.7.2 o superior
- **Sistema operativo:** Windows 10 o Windows 11

Pasos para la instalación

a. Descargar el proyecto

Obtenga la solución completa del sistema que incluye los formularios de Windows Forms y el archivo .sln de Visual Studio. Puede copiar el proyecto desde una unidad USB, carpeta compartida o repositorio privado, según el método de distribución utilizado.

[\(AGREGAR GITHUB REPOSITORIO\)](#)

b. Instalar SQL Server Management Studio

Descargue e instale **SQL Server Management Studio (SSMS)** desde el sitio oficial de Microsoft. Este programa permitirá ejecutar y gestionar la base de datos del sistema.

c. Restaurar la base de datos

Abra SSMS y cree una nueva base de datos con el nombre `Ligia_SalonPTC2`. Ejecute el script SQL proporcionado para crear todas las tablas, relaciones y registros iniciales del sistema. Este script debe incluir la estructura completa del sistema: roles, usuarios, clientes, servicios, citas, facturas y detalles.

d. Abrir la solución en Visual Studio

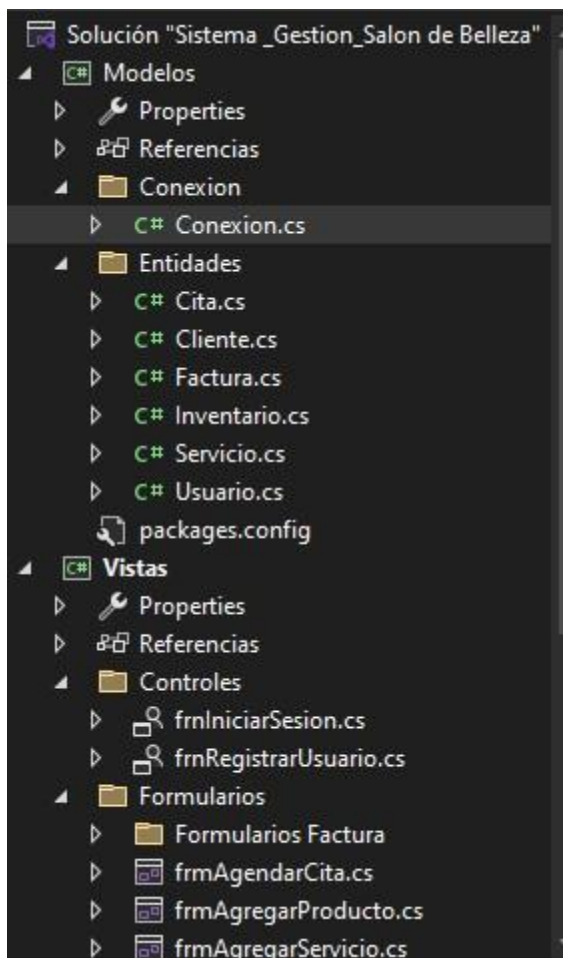
Abra Visual Studio 2022 y cargue el archivo **.sln** del proyecto.

Verifique que todas las referencias estén correctamente configuradas y que el proyecto compile sin errores.

e. Configurar la cadena de conexión

Localiza el archivo de configuración del sistema (Conexión.cs el archivo donde se define la cadena de conexión).

Modifique la cadena de conexión para que apunte al nombre del servidor SQL local.



El valor **DESKTOP-NOMBRE\\SQLEXPRESS** debe ser reemplazado por el nombre real del servidor SQL en la máquina donde se instala el sistema.

Ejemplo:

```
public class Conexion
{
    private static string servidor = "DESKTOP-NOMBRE\\SQLEXPRESS";
    private static string baseDeDatos = "Ligia_SalonPTC2";

    public static SqlConnection Conectar()
    {
        try
        {
            string cadena = $"Data Source={servidor};Initial Catalog={baseDeDatos};Integrated Security=true;TrustServerCertificate=True;";
            SqlConnection conexion = new SqlConnection(cadena);
            conexion.Open();
            return conexion;
        }
        catch (Exception ex)
        {
            MessageBox.Show("No se pudo conectar al servidor: " + ex.Message, "Error de Conexión",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            return null;
        }
    }
}
```

f. Ejecutar el sistema

Compile y ejecute el proyecto desde Visual Studio.

El sistema se abrirá en modo escritorio, mostrando los formularios de inicio de sesión, gestión de clientes, servicios, inventario, citas y facturación.

g. Validar funcionamiento

Verifique que todas las funcionalidades estén operativas:

- Registro y autenticación de usuarios
- Visualización y edición de clientes
- Gestión de servicios y productos
- Creación de citas y generación de facturas

- Visualización de reportes y detalles

Referencias

Microsoft. (s.f.). C# documentation. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/csharp/>

Microsoft. (s.f.). Windows Forms documentation. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/>

Microsoft. (s.f.). SQL Server Express. Microsoft. <https://www.microsoft.com/es-es/sql-server/sql-server-downloads>

Microsoft Azure. (s.f.). Azure SQL Database. Microsoft Azure. <https://azure.microsoft.com/es-es/products/azure-sql/database>

Mozilla. (s.f.). HTML5. MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTML>

Microsoft. (s.f.). Visual Studio. Microsoft. <https://visualstudio.microsoft.com/es/>

Microsoft. (s.f.). Visual Studio Code. Microsoft. <https://code.visualstudio.com/>

Microsoft. (s.f.). SQL Server Management Studio (SSMS). Microsoft Learn. <https://learn.microsoft.com/es-es/sql/ssms/download-sql-server-management-studio-ssms>

Figma. (s.f.). Figma design tool. Figma. <https://www.figma.com/>

Diagrams.net. (s.f.). Draw.io. <https://app.diagrams.net/>

Lucid Software. (s.f.). Lucidchart. <https://www.lucidchart.com/>